SEVA-SADHANA

Web App Implementation Plan

Goal: build a mobile-first web app that turns the Seva-Sadhana philosophy (Need, Seva, Empowerment, Self-Reliance, Contribution, Awareness) into a working product, starting with the MVP loop **Sankalp, Radar, Moment, Mirror**, with dignity built into the data model, not added afterwards.

1. Recommended approach

Build a **mobile-first Progressive Web App (PWA)**. The original document specifies Flutter, but for a web app a PWA gives one codebase, installability on phones and easy sharing by link, which matters for a community project. The planned Python/FastAPI/PostgreSQL backend stays unchanged.

Layer	Choice	Why
Frontend	Next.js (React + TypeScript), Tailwind, PWA support	Fast to build, good routing, installable on phones
Backend	FastAPI (Python)	Matches the original plan; Python suits the later AI layer
Database	PostgreSQL + PostGIS	Relational data plus 'opportunities near me' queries
Auth	Email/phone OTP (or Firebase Auth / Auth0)	Familiar to Indian users; avoids storing passwords
Maps	OpenStreetMap + Leaflet (free) or Google Maps	Cost vs. polish trade-off
Notifications	Web Push + email (later WhatsApp/SMS)	Gentle reminders for Sankalp and Mirror
AI (Phase 2)	Claude API	Opportunity understanding and reflection summaries
Hosting	Vercel (frontend); Railway/Render/Fly.io (API + DB)	Low cost, easy deploys

2. System architecture

```
Browser / PWA (Next.js)
    |  REST/JSON (JWT)
FastAPI backend: Auth, Profiles, Opportunities, Matching,
                 Moments, Reflections, Circles, Consent
    |
PostgreSQL + PostGIS
    |  (Phase 2+)
AI Seva Engine (LLM calls behind a service layer)
    |
Maps / Push notifications / Email
```

Keep matching and AI logic behind a **service interface** (`RecommendationService`). The MVP implementation is rule-based; AI can be swapped in later without touching the rest of the app.

3. Core data model

Entity	Key fields
users	id, name, auth identifier, language, city/area, created_at
contribution_profiles	user_id, time_availability, learning_interests, resources_offered
skills / user_skills	skill, level, **wants_to_learn** flag (removes the giver/receiver identity)
sankalps	user_id, quality (patience, listening, humility...), date
opportunities	title, description, category, location (geography point), time needed, urgency, skills needed, status, source (user / organization), visibility
consents	user_id, opportunity_id, what is shared, granted_at, revoked_at
seva_moments	user_id, opportunity_id (nullable for self-reported acts), sankalp_id, duration, **is_silent**, completed_at
reflections	moment_id, learned, listened, expected_recognition, do_differently (private by default)
seva_circles / circle_members	group, recurring activity, schedule, members
seva_chain_links	from_user, to_user/community, moment_id (Phase 3)
organizations	NGOs, colleges, verified status (Phase 4)

Dignity rules in the schema: no *points*, *rank* or *total_hours_public* columns, and no field labelling anyone a 'beneficiary'. Recipients are never shown as profiles. An opportunity describes the **activity**, not the person.

4. MVP scope (matches section 11 of the source document)

Screens and routes

Route	Purpose
/welcome	Onboarding: 'I want to practise Seva', with no role selection
/sankalp	Choose today's quality (8 to 10 cards, each with a one-line meaning)
/home	Today's Sankalp plus 3 to 5 nearby opportunities
/radar	List and map of 5 to 10 opportunities; filter by category, time, distance
/opportunity/[id]	Detail page with a clear consent and dignity checklist
/profile	Contribution Profile: skills, time, learning interests
/moment/[id]	Start and complete a Seva Moment; option for Silent Seva
/mirror/[momentId]	Reflection form (3 short prompts, skippable)
/journey	Personal timeline of Sankalps, Moments and reflections (no scores)
/circle	Create or join a simple circle for one recurring activity

Core backend endpoints

```
POST /auth/otp/request, /auth/otp/verify
GET/PUT /me/profile
POST /sankalps              GET /sankalps/today
GET  /radar?lat=&lng=&category=&sankalp=
GET  /opportunities/{id}
POST /opportunities/{id}/consent
POST /moments               PATCH /moments/{id}/complete
POST /moments/{id}/reflection
GET  /journey
POST /circles               POST /circles/{id}/join
```

Rule-based matching (MVP)

Score each opportunity per user with a plain weighted sum in one function, so it is easy to explain and tune:

- **Category fit** with the chosen Sankalp (e.g. 'listening' maps to Companionship and Medical accompaniment)
- **Skill overlap** with the profile, or a learning interest the activity supports
- **Time fit**: the activity takes less time than the user has free
- **Distance** using PostGIS `ST_DWithin`
- **Urgency** as a small boost

Show the *reason* ('because you chose Patience and this involves teaching') rather than a numeric score.

5. Build phases and timeline

Assumes 1 to 3 developers.

Phase	Duration	Deliverables
0. Foundations	Weeks 1-2	Repo, CI, DB schema, auth, design system, Sankalp and Mirror copywriting
1. MVP	Weeks 3-8	Screens 1 to 9, rule-based Radar, admin panel to add opportunities
1b. Pilot	Weeks 9-10	Simple Seva Circle, PWA install, push reminders, pilot with one NGO or college group
2. AI layer	Weeks 11-14	Natural-language opportunity entry, reflection pattern summaries
3. Community	Weeks 15-20	Seva Chain, Seva Mandala (aggregate dashboards), richer Circles
4. Institutions	Later	Organization accounts, verification, CSR reporting
5. Emergency Seva	Later	Rapid coordination mode (needs reliability work first)

6. The AI layer (Phase 2)

- **Opportunity understanding:** a user types 'An elderly neighbour needs help linking her UPI app.' The LLM returns structured JSON (category: Digital Seva, skills, time estimate, urgency) that pre-fills the opportunity form for human confirmation.
- **Reflection assistance:** summarise patterns across reflections and suggest a next Sankalp. Phrase it as a mirror, never a verdict.
- **Community analytics:** aggregate-only queries (counts by area and category) with a minimum group size before any number is shown, so no individual is identifiable.
- **Data minimisation:** send the AI only the text it needs, never names or contact details; log prompts and outputs without personal data.

7. Privacy, safety and dignity checklist

- Consent is a **recorded event**, not a checkbox. Contact details are released only after both sides agree, and consent can be revoked.
- Reflections are private by default and never shown to other users.
- Silent Seva moments are excluded from every public or circle-level view.
- No leaderboards, streak-shaming or public hour counts, including in the database and API.
- In-person safety: first meetings in public or organization-hosted places, a report/block button, and organization verification for medical and accessibility categories.
- Medical Support stays **non-clinical** by design, with clear copy and no health data stored.
- Comply with India's DPDP Act: purpose-limited collection, a privacy notice, and account and data deletion. Get a legal review before launch.
- Moderation: opportunities from unverified users are reviewed before appearing on Radar.

8. The cold-start problem

An empty Radar kills the app. Before launch:

- Partner with **one pilot community** (a college NSS unit, a local NGO or a Delhi neighbourhood group).
- Seed 20 to 30 real opportunities manually through the admin panel.

- Add recurring 'standing' opportunities (weekly teaching, digital literacy desk) so something is always available.
- Allow **online** opportunities (mentoring, form help) so location is not a hard limit.

9. Testing and measuring success

- **Tests:** unit tests for the matching function, API tests for consent flows, and Playwright end-to-end tests for Sankalp, Radar, Moment, Mirror.
- **Pilot metrics that fit the philosophy:** share of Moments followed by a reflection, share of people who return for a second Sankalp, Circles that continue after 4 weeks, and people who move from receiving to contributing.
- **Avoid** engagement-maximising metrics such as time-on-app.
- **Qualitative:** short interviews with pilot users about whether the app felt dignified.

10. Risks and mitigations

Risk	Mitigation
Empty Radar	Pilot partner and seeded opportunities
Reflection feels like homework	Keep Mirror to 3 short prompts; allow skipping
Misuse or safety incidents	Verification, reporting, public-place guidance
Scope creep into 9 modules at once	Strictly follow the MVP list; hide unbuilt modules
'Role-free' confuses users	Test onboarding wording early with real users

11. First two weeks: concrete next steps

1. Set up a monorepo (`/web` Next.js, `/api` FastAPI) with Docker Compose for Postgres/PostGIS.
2. Write the schema migrations (Alembic) for the entities in section 3.
3. Implement OTP auth and the Contribution Profile.
4. Write the copy for the Sankalp qualities and the Mirror prompts. This is core to the product; do it before heavy UI work.
5. Build the admin page for adding opportunities so seeding can start in parallel.

> **Closing thought:** the app should act as a mirror and guide, not a judge or scoreboard. Whenever a feature decision is unclear, ask: does this reduce anyone to a giver or a receiver, or rank them? If yes, leave it out.